

Algorithmes et Pensée Computationnelle

Algorithmes de recherche - Exercices avancés

Le but de cette séance est de se familiariser avec les algorithmes de recherche. Dans cette série d'exercices, nous manipulerons des listes et collections en Java et Python. Nous reviendrons sur la notion de récursivité et découvrirons les arbres de recherche. Au terme de cette séance, l'étudiant sera en mesure d'effectuer des recherches de façon efficiente sur un ensemble de données.

Le code présenté dans les énoncés se trouve sur Moodle, dans le dossier `advanced`.

1 Recherche binaire

Question 1: (🕒 15 minutes) Recherche binaire - plus proche élément (Python)

Soit une liste d'entiers **triés** `L` et un entier `e`. Écrivez un programme retournant la valeur dans `L` la plus proche de `e` en utilisant une recherche binaire (binary search).

```
1 def plus_proche_binaire(liste,n):
2     #complétez ici
3
4 if __name__ == '__main__':
5     L = [1, 2, 5, 8, 12, 16, 24, 56, 58, 63]
6     e = 41
7     print(plus_proche_binaire(L,e))
8     # Résultat attendu : 56
```

💡 Conseil

Pensez à définir des variables `low` et `high` délimitant l'intervalle de recherche.

>_ Solution

```
1
2 def plus_proche_binaire(liste, e):
3     low = 0
4     high = len(liste) - 1
5
6     if(e >= liste[high]):
7         return liste[high]
8     if(e <= liste[low]):
9         return liste[low]
10
11     while low <= high: # 0<10 and true puis 6<10 and true, etc.
12         mid = (low + high) // 2 # mid = 5 --> 16 in list
13         print(mid)
14         if e > liste[mid]: # 41>16
15             low = mid + 1 # min = 5+1=6
16         elif e < liste[mid]:
17             high = mid - 1
18         else:
19             return liste[mid]
```

>_ Solution

```
21     if abs(liste[low]-e) < abs(liste[high]-e) :
22         return liste[low]
23     else:
24         return liste[high]
25
26
27 if __name__ == '__main__':
28     L = [1, 2, 5, 8, 12, 16, 24, 56, 58, 63]
29     e = 61
30     print(plus_proche_binaire(L, e))
```

Question 2: (🕒 20 minutes) Recherche dans une matrice (Python)

Considérez une matrice ordonnée m et un élément l .

Pour rappel, une matrice ordonnée répond aux critères suivants :

$m[i][j] \leq m[i+1][j]$ (une ligne va du plus petit au plus grand)

$m[i][j] \leq m[i][j+1]$ (une colonne va du plus petit au plus grand)

Écrivez un algorithme qui retourne la position de l'élément l dans m . Si l n'est pas présent dans m alors, votre programme retournera $(-1, -1)$.

>_ Exemples

Exemple 1 : si $m = [[1, 2, 3, 4], [4, 5, 7, 8], [5, 6, 8, 10], [6, 7, 9, 11]]$ et que $l=7$. Nous souhaitons avoir la réponse $(1, 2)$ OU $(3, 1)$ (l'une des deux, pas besoin de retourner les deux résultats).

Exemple 2 : si $m = [[1, 2], [3, 4]]$ et que $l=7$. Nous souhaitons avoir la réponse $(-1, -1)$ car 7 n'est pas dans la matrice m .

```
1 def recherche_matricielle(m, l):
2     #complétez ici
3
4
5 m = [[1, 2, 3, 4], [4, 5, 7, 8], [5, 6, 8, 10], [6, 7, 9, 11]]
6 l = 7
7 recherche_matricielle(m, l)
```

💡 Conseil

Pour cet exercice, il est nécessaire d'utiliser des boucles `for` imbriquées, c'est-à-dire : une boucle `for` dans une autre boucle `for`. Cela permet de parcourir tous les éléments d'une liste (ou d'un tableau) à deux dimensions (dans notre cas une matrice).

>_ Solution

Python :

```
1 def recherche_matricielle(m,l):
2     #complétez ici
3     for ligne in range(len(m)): # "Pour chaque ligne de la matrice" ou
        "Pour chaque liste de la liste"
4         for colonne in range(len(m[ligne])): # "Pour chaque colonne de
            la matrice" ou "Pour chaque élément de la liste"
5             if m[ligne][colonne] == l:
6                 return (ligne,colonne)
7     return (-1,-1)
8
9 if __name__ == '__main__':
10     m=[[1,2,3,4],[4,5,7,8],[5,6,8,10],[6,7,9,11]]
11     l=61
12     print(recherche_matricielle(m,l))
```